

School of Computing and Information Systems
COMP90038 Algorithms and Complexity

Assignment 1 (2026 Semester 1)

Due date: **11:59pm April 16 (Thursday) 2026**

Marks. This assignment has in total 15 marks and is worth 15% of the total subject mark.

Objectives. To improve your understanding of the time complexity of algorithms and recurrence relations. To develop problem-solving and design skills. To improve written communication skills; in particular, the ability to present algorithms clearly, precisely and unambiguously.

Difficulty. The number of * indicates the difficulty of a problem: (*) means *a bit* challenging, and (**) means *very* challenging. There is **no** (**) -problem in this assignment.

Dictionary Search

Recall that given a set S of n integers sorted in ascending order and stored in an array A of length n , and a target search integer v , the goal of the Dictionary Search problem is to decide whether $v \in S$. In class, we introduced the *binary search* algorithm for solving this problem.

Problem 1. [3 Marks] Now, let us consider a *variant* of the above *binary search* algorithm, called *1/5-search*. Specifically, the *1/5-search* algorithm works as follows:

The base case. If $n \leq 5$, check if v is one of the *only* at most 5 integers in the array in a brute-force manner.

The inductive case. When $n > 5$, do the following:

- compare v to the integer s stored in the position of $1/5$ of the length of the array (for example, if the current array is of length 10, then we pick the integer stored at position $\frac{1}{5} \times 10 = 2$ in the array);
- if $v = s$, return *yes* and done;
- otherwise,
 - if $v < s$, recursively solve the problem on the “left” part of the array *before* s ;
 - if $v > s$, recursively solve the problem on the “right” part of the array *after* s .

Prove that the running time complexity of *1/5-search* algorithm is bounded by $O(\log n)$. **Hint:** You will need to define the cost recurrence and then solve it.

Problem 2. [4 Marks] Now, we further consider an extension of the above *1/5-search* algorithm, which is called α -search algorithm for some arbitrary parameter $0 < \alpha \leq 1/2$. As a result, the *1/5-search* algorithm is a special case of the α -search with $\alpha = 1/5$. However, note that, in this Problem 2, α should **not** be considered as a *constant*. That means, α can be as small as some function of the *original input size* n , e.g., $\alpha = \frac{1}{\log n}$. Once the value of α is set, its value does not change through out the whole algorithm. Hence, α should appear in the time complexity.

To avoid confusion, although it is highly similar to that of the *1/5-search*, we put the full description of the α -search algorithm below:

The base case. If $n \leq \lfloor \frac{1}{\alpha} \rfloor$, check if v is one of the *only* at most $\lfloor \frac{1}{\alpha} \rfloor$ integers in the array in a brute-force manner.

The inductive case. When $n > \lfloor \frac{1}{\alpha} \rfloor$, do the following:

- compare v to the integer s stored in the position of α of the length of the array, i.e., the position at $\lfloor \alpha n \rfloor$.
- if $v = s$, return *yes* and done;
- otherwise,
 - if $v < s$, recursively solve the problem on the “left” part of the array *before* s ;
 - if $v > s$, recursively solve the problem on the “right” part of the array *after* s .

Answer the follow questions.

- (a) [1 Mark] What is the running time complexity of the base case in the α -search algorithm? Note: α should appear in the complexity.
- (b) [3 Marks] Prove that the running time complexity of the α -search algorithm is bounded by $O(\frac{1}{\alpha} + \log_{\frac{1}{1-\alpha}} n)$.

Two-Sum

Problem 3. [8 Marks] Consider a set S of n integers sorted in *ascending* order and stored in an array A of length n ; given a target sum value k , the goal of the *Two-Sum* problem is to decide if there exists a pair of integers $a_1 \in S$ and $a_2 \in S$ such that $a_1 + a_2 = k$, where a_1 and a_2 can be the same integer, i.e., $a_1 = a_2$. Answer the following questions.

Note: Using either pseudo code or descriptive language (just like the algorithm descriptions in Problem 1 and 2) to describe algorithms would be fine, as long as your algorithm descriptions are correct, clear, rigorous and non-ambiguity.

- (a) [2 Marks] Describe a brute-force algorithm to solve the above *Two-Sum* problem, and analyse the time complexity of your algorithm.
- (b) [3 Marks] Describe an algorithm that can solve the *Two-Sum* problem in $O(n \log n)$ worst-case time. You will need to prove the upper bound of the time complexity of your algorithm.
- (c) (*) [3 Marks] Describe an algorithm that can solve the *Two-Sum* problem in $O(n)$ worst-case time. You will need to prove the *correctness* and the upper bound of the time complexity of your algorithm. Note: algorithm description, correctness proof, and running time analysis take 1 mark each.

Submissions

You should lodge your submission for Assignment 1 via the LMS (i.e., Canvas). *You must identify yourself in your file.* Poor-quality scans of solutions written or printed on paper will *not* be accepted. There are scanning facilities on campus, not to mention scanning apps for smartphones etc. Solutions generated directly on a computer are of course acceptable. Submit *one* file:

- A *assignment1.pdf* file, comprising your solutions to the questions in the assignment.

Administrative Issues

When is late? What do I do if I am late? The due date and time are printed on the front of this document. **The late penalty for non-exam assessment is *two* marks per day (or part thereof) overdue.** Requests for extensions or adjustment must follow the University policy (the Melbourne School of Engineering “owns” this subject), including the requirement for appropriate evidence.

Late submissions should also be lodged via the LMS, but, as a courtesy, please also email the subject coordinator, Junhao Gan, when you submit late. If you make both on-time and late submissions, please consult the subject coordinator as soon as possible to determine which submission will be assessed.

Individual work. You are reminded that your submission for this Assignment is to be your own individual work. Students are expected to be familiar with and to observe the University’s Academic Integrity policy <http://academicintegrity.unimelb.edu.au/>. For the purpose of ensuring academic integrity, every submission attempt by a student may be inspected, regardless of the number of attempts made.

Students who allow other students access to their work run the risk of also being penalized, even if they themselves are sole authors of the submission in question. **By submitting your work electronically, you are declaring that this is your own work.** Automated similarity checking software may be used to compare submissions.

You may re-use code and materials provided by the teaching staff, and you may refer to resources on the Web or in published or similar sources. Indeed, you are encouraged to read beyond the standard teaching materials. However, *all* such sources *must* be cited fully and, apart from code and materials provided by the teaching staff, you must *not* copy code or any materials.

Moreover, No AI assistant tool, e.g., ChatGPT and the like, is allowed to use for completing this assignment in any form.

Finally. *Despite all these stern words, we are here to help!* There is information about getting help in this subject on the LMS pages. Frequently asked questions about the Assignment will be answered in the LMS discussion group.